

Warm Up

How do we make sure we can roll back mistakes?

Programming is challenging. One wrong variable and we could mess up our whole data pipeline.

Objectives for today:



Git & GitHub

Learn what Git, GitHub, and Codespaces are and how they form your toolkit.



Jupyter Notebooks

Explore the browser-based workstation further for writing and running code.



Hands-on Coding

Apply foundational Python concepts:

- Variables
- Flow Control (Loops)
- Functions

1. What is Git?

Git is the first component in our toolkit—a powerful system for safe and efficient coding.



The "Time Machine" for Code

- **Version Control System:** Safely update code and roll back to working versions to debug errors.
- **Change Tracking:** Keeps a complete history of every modification made to your project.
- **Collaboration:** Essential for working on complex projects or sharing code with teammates.

"It's like the coding equivalent of a Google Doc that automatically saves every previous version of your work."

2. What is GitHub?

GitHub is like an online home for your code, using Git to store and organize projects in the cloud.



It allows you to:

- **Safe Cloud Storage:** Save work safely online, similar to Google Drive for code.
- **Global Collaboration:** Work together with classmates or developers worldwide.
- **Professional Portfolio:** Build and showcase your programming portfolio to others.

Used by professionals and companies everywhere to manage code and projects at scale.

3. What is GitHub Codespaces?

GitHub Codespaces is a coding environment in the cloud—access a full development setup from your browser.



Why It's Useful:

- **Code Anywhere:** Work anytime with just internet access—no local installations required.
- **Zero Setup:** Perfect for students to avoid complicated software configuration on their own computers.
- **Jupyter Ready:** Fully supports Jupyter Notebooks for writing and running Python code seamlessly.

This finally brings us to the last topic: a browser-based "Time Machine" and workstation for your projects.

Summary



Git

Helps you track changes to your code (like a time machine).



GitHub

The online home where you store and share your code.



GitHub Codespaces

An online coding environment to start right away, no downloads needed.

"These tools will help you build, share, and manage your coding projects throughout this course and beyond."

4. What is a Jupyter Notebook?

A versatile tool for writing, testing, and running code combined with rich text explanations.



Key Features:

- **Interactive Cells:** Switch between CODE cells for programming and MARKDOWN cells for written text.
- **Narrative Coding:** Explain your logic and share insights directly alongside your code.
- **Industry Standard:** Widely used in professional data science to write, test, and document work.

"Jupyter Notebooks strive to have a dedicated, organized location for both your logic and your writing."

Summary: Jupyter Notebooks



1. What is it?

A tool to write and run code in small chunks.

Combines code, text, and visuals in one place.

An interactive digital notebook for instant results.



2. Why is it useful?

Perfect for experimenting with instant feedback.

Enables explanations alongside live code.

The standard tool for data analysis and sharing.



3. How does it work?

Organized into specialized "cells".

Code cells run logic; Text cells hold notes.

Output is displayed directly below the code cell.

Activity: First Dip into Coding

Now that we've created our first Jupyter Notebooks, let's dive into markdown and coding.



A Note on Being a Good Learner

In many activities, you'll be asked to type phrases. **Avoid copying and pasting.**

- Physically typing helps you learn the nuances and common mistakes of code.
- While tedious at first, this manual effort is a fundamental part of mastering programming.
- Education is achieved through active participation, not shortcuts.

"If you want to circumvent the education process, you won't achieve at the levels you want in life."

Activity: First Dip into Coding

Code Practice 1 - Print

Instructions:

1. **To start:** Click **Markdown** at the top of your Jupyter Notebook. Type: *'Unit 1 Problems'* and hit **option + return**. Notice how the text changes from an active - editable cell to just background text.
2. Hover your cursor over the bottom of the markdown cell and click **Code** to add a new code cell.
3. Type the following: `print("Hello world!")`
4. Press **control + enter** (or shift + enter).

*It should display **Hello world!** below your code cell.*

You can learn shortcuts to make this faster at this [link](#).

Reflection: Your First Python Code



Congratulations! You've officially started your programming journey.

Running "Hello World" is a time-honored tradition for every programmer learning a new language.



Key Takeaways: The `print()` Function

The **print** function is a built-in tool in Python used to display text. Here's what you should know about using it:

- **Quotes are Mandatory:** They differentiate **strings** (text) from **variables**.
- **Flexibility:** You can use either single quotes (`'...'`) or double quotes (`"..."`). Both work identically in Python.
- **Dynamic Updates:** Changing the letters inside the quotes results in a new message instantly.

You'll get more practice with Variables and Strings later in this course!

Activity: Math Operations

Code Practice 2 - Math Operations

Instructions & Practice:

1. Let's try out a few other programming exercises to get used to the different things Python allows us to do.
2. In a new code cell type `5 + 7` and press **control + enter** again.
3. Notice that the output is the result of the math equation.
4. Python can evaluate many math expressions but you'll have to use the correct notation. I've linked to a few different operations and their notations [here](#).

These can be helpful in the future when you need to perform operations on data sets.

Activity: Variables in Python

Code Practice 3 - Variables

Instructions & Practice:

Just like in math, we can have variables in programming to store information. Try the following steps:

1. In a new code cell, type the following two lines:

```
name = "type your name"  
name
```

1. Run the cell using **control + return** (or the run button). Your name should appear as the output.
2. Now, change the second line to: `print("Hello " + name + "!")`
3. Run it again. The variable **name** is replaced by your actual name in the output!

This concept is essential for chatbots, applications, and managing data sets.

Activity: Flow Control

Code Practice 4 - Loops

Instructions & Practice:

1. Hover your cursor over the bottom of your previous cell and click **Code** to add a new code cell.

Type the following code precisely, noting the indentation on the second line:

```
2. for i in range(5):  
    print("This is line ", i + 1)
```

3. Press **control + enter** to run the cell.

Loops allow you to repeat actions efficiently, which is a fundamental concept in flow control.

Concept: Indentation & Flow Control

Understanding Python Program Structure

Key Takeaways



Indented Code Blocks

Python uses indentation to define code structure. Certain statements cause these blocks to occur automatically.



The Colon (:) Trigger

Generally, any line of code that ends in a colon creates a new block of code on the next line.



Control Flow Statements

Statements like "for" tell Python which code to execute first, allowing it to move beyond simple top-to-bottom execution.

While code normally executes top-to-bottom, Control Flow Statements allow us to skip or repeat sections based on logic.

Concept: The 'for' Loop & range()

Breaking Down the Output and Logic

What is a for loop?

A for loop is a repeated instruction that tells the computer to do the same thing repeatedly, with slight variations each time.

The range() function

range() is a built-in function that lists numbers starting at 0 up until the number of your choosing.

This allows us to display lists or change sentences dynamically.

Code Trace Analysis

In our example, the loop changes `i` through the following values:

- 0, 1, 2, 3, and 4

Execution logic:

- 1 is added to each value of `i`
- Result is appended to "This is line"